



Redes Neurais

UNIDADE 02

Rede neural MLP (Multi-Layer Perceptron)

| Instalação do ambiente de desenvolvimento integrado (IDE)

Para darmos início aos nossos estudos de redes neurais, precisamos, antes de tudo, de ferramentas computacionais. Durante a disciplina, iremos utilizar a IDE conhecida como Spyder 5, utilizando a linguagem de programação Python 3.1 (ou superior).

Portanto, confira a seguir um breve tutorial de como fazer o *download* e a instalação (caso você ainda não possua esta IDE) em seu computador pessoal.

1. Inicialmente, baixe do site www.anaconda.com/products/individual a versão mais recente do Anaconda.
2. Instale-o em seu computador, seguindo os passos do instalador. Utilize a opção “Instalar apenas para mim”.
3. Abra o *software* Anaconda Navigator.
4. Na janela principal, clique em “*Enviroments*”, no menu à esquerda.
5. Clique no botão “*Create*”, escolha um nome para seu novo ambiente e selecione a versão mais atual do pacote “Python”. Clique em “*Create*” nesta janela para criar o ambiente.
6. Ainda na guia “*Environments*”, digite tensorflow no campo de busca (canto superior direito) e instale os seguintes pacotes:
 - a. keras
 - b. tensorboard
 - c. tensorflow
 - d. tensorflow-estimator
 - e. tensorflow-intel
 - f. tensorflow-io-gcs-filesystem
 - g. transformers
7. Retorne para a guia “*Home*”.
8. Na guia “*Home*”, selecione o pacote Spyder, clique para instalar e aguarde até a finalização da instalação.
9. Agora o botão “*Install*” do pacote Spyder mudou para “*Launch*”. Clique em “*Launch*” para abri-lo.
10. Na tela inicial do Spyder, clique em “*View*”, selecione “*Window layouts*” e depois, “*Horizontal split*”.

Feito! A sua IDE Spyder está pronta para uso. Caso tenha alguma dificuldade com a instalação, entre em contato com o professor-tutor da disciplina, ele irá ajudá-lo da melhor forma possível. Mas, no momento, ainda não iremos utilizar a IDE, pois, primeiramente, vamos estudar as particularidades e funcionalidades (conceitos gerais e técnicos) das redes neurais MLP.

E caso precise abrir o Spyder em momento futuro novamente, basta abrir o Anaconda Navigator (agora já instalado), e na guia “*Home*” clicar em “*Launch*” no pacote Spyder, assim como você fez no passo 9.

| REDE NEURAL MLP (MULTI-LAYER PERCEPTRON)

Conceitos Gerais

As redes neurais de MLP (em português, podemos chamá-las de redes neurais de múltiplas camadas) fazem parte da classe de modelos de aprendizagem de máquina. Como sabemos, tais modelos são baseados na estrutura e funcionamento do cérebro humano, simulando conexões (sinapses) entre neurônios.

Cada um dos neurônios da camada de entrada desta rede neural é chamado *perceptron*. Esses neurônios recebem dados de entrada e aplicam uma função de ativação, gerando uma saída. A camada de saída, com a utilização de algoritmos de aprendizado, realiza a retropropagação do erro, ajustando os pesos das conexões entre neurônios para reduzir o erro e melhorar o desempenho de previsão do modelo.

Aplicações

As redes neurais MLP são utilizadas principalmente para realizar trabalhos complexos que envolvem problemas como:

- Classificação: para distinguir e classificar elementos de um conjunto de dados com base em suas características individuais.
- Regressão: para realizar previsões numéricas sobre uma característica de um elemento, de acordo com outras características deste mesmo elemento.

- Reconhecimento de padrões: para identificar repetições sazonais de uma base de dados ou correlações entre variáveis que se repetem ao longo do tempo.

Arquitetura

As redes neurais MLP possuem uma arquitetura totalmente conectada, também chamada, no inglês, de “*fully connected*”, em que os neurônios de uma determinada camada estão conectados com todos os neurônios das camadas adjacentes (anterior e posterior), como mostra a Figura 1.

Ainda observando a Figura 1, cada círculo representa um neurônio, cada coluna de neurônios é tida como uma camada, as setas que conectam os neurônios são os pesos (ou sinapses). De uma maneira simplificada, podemos afirmar que os cálculos realizados, utilizando-se dos valores dos neurônios de uma determinada camada e os valores dos pesos, são “disparados” para a camada seguinte de neurônios, sobrepondo valores já existentes. Este processo se repete até que os neurônios da camada de saída sejam ativados.

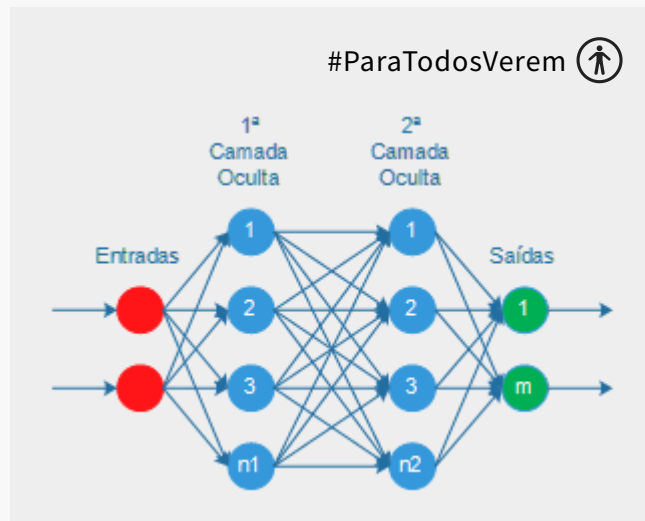


Fig. 1 – Estrutura genérica de uma Rede Neural MLP. Fonte: o autor, 2023.

Para problemas de classificação, em que o objetivo é atribuir uma classe distinta ao elemento que está sendo testado, temos um número de neurônios na camada de saída igual ao número de classes existentes. Por exemplo, se precisamos identificar números de 0 a 9, num conjunto de imagens que contém apenas um dígito numérico cada, teremos 10 classes (os próprios números de 0 a 9) e, com isso, teremos também 10 neurônios na camada de saída. A ativação destes neurônios ocorrerá de forma única, ou seja, apenas 1 neurônio será ativado, de acordo com o elemento de entrada e com a identificação que o modelo da rede neural realizou.

Neste caso, após realizado o treinamento da rede, se a alimentarmos com uma imagem contendo o número 4, por exemplo, caso o modelo esteja bem treinado e apresente boa precisão, espera-se que o quarto neurônio da camada de saída seja ativado, nos dando como resposta o número 4; mostrando, com isso, que o modelo identificou corretamente o número contido na imagem que o alimentou.

Porém, para que a rede identifique corretamente o elemento de entrada, é necessário que ela seja desenvolvida de forma correta e, além disso, que tenha seus parâmetros ajustáveis configurados de acordo com o problema a ser resolvido, e de acordo com os dados de treinamento.

Veremos, então, quais são esses parâmetros ajustáveis e qual a função de cada um deles na rede.

Quantidade de neurônios nas camadas da Rede Neural MLP

- Camada de entrada: a quantidade de neurônios na camada de entrada será igual à quantidade de dados existentes em um único elemento da base de dados. Por exemplo, se quisermos identificar números em figuras de tamanho 28x28 (em *pixels*), podemos utilizar os próprios pixels da figura como dados de entrada. Ou seja, teremos um total de 784 neurônios na camada de entrada.
- Camada de saída: como mencionado, a quantidade de neurônios da camada de saída será igual à quantidade de classes existentes para se classificar os elementos da base de dados. Por exemplo, podemos ter milhões de figuras contendo números que necessitam de identificação, mas se cada uma delas possui apenas um único dígito, fica evidente que o número de classes será igual a 10 (cada uma representando um número de 0 a 9). Com isso, a quantidade de neurônios na camada de saída também será igual a 10, cada um também representando um número (uma classe), de 0 a 9.
- Camada oculta: embora possamos ter mais do que uma camada oculta, afinal esse tipo de rede possui múltiplas camadas, neste primeiro momento iremos considerar a utilização de apenas uma. Para que possamos determinar com melhor precisão o número de neurônios desta camada, em geral, precisamos treinar o modelo várias vezes (com diferentes quantidades de neurônios na camada oculta) e

compreender qual é o comportamento dele em relação aos dados de entrada, verificando o desempenho da rede a cada treinamento. Entretanto, determinar corretamente o número de neurônios de uma camada, bem como a quantidade de camadas ocultas, trata-se de um dos maiores desafios durante o desenvolvimento de uma rede neural (e não apenas da MLP). Mas, como ponto de partida, podemos utilizar um procedimento básico, o que não se trata de uma regra, mas que pode ajudar num primeiro momento: somam-se as quantidades de neurônios das camadas de entrada e de saída e divide-se o resultado por 2. Ou seja, se temos 784 neurônios de entrada e mais 10 de saída, teremos um total de 794 neurônios, e a metade disso é 397. Então, inicialmente, podemos definir esta quantidade para a camada oculta. Entretanto, vale lembrar que precisamos treinar o modelo e verificar o desempenho da rede executando testes após o treinamento. Definir 397 neurônios para a camada oculta não é garantia de que teremos o melhor modelo possível, mas é um bom ponto de partida.

Função de ativação

As funções de ativação são algoritmos que realizam cálculos matemáticos internos para disparar os sinais entre neurônios conectados de duas camadas.

As redes neurais MLP possuem várias e diferentes funções de ativação que podem ser utilizadas:

- ReLU (Rectified Linear Unit): retorna 0 se o valor de entrada for negativo, e retorna o próprio valor de entrada em caso contrário.
- Sigmoid: conhecida como função logística, retorna o valor $1 / (1 + e^{(-x)})$ para uma entrada x . Ou seja, mapeia probabilidades para o neurônio no qual foi aplicada.
- Tangente hiperbólica: retorna $(e^x - e^{(-x)}) / (e^x + e^{(-x)})$ para uma entrada x , mapeando qualquer valor entre -1 e 1, assumindo valores negativos para entradas negativas e valores positivos para entradas positivas. Bastante útil para mapear entradas com valores simétricos sobre o eixo X .
- Softmax: mapeia vetores de números reais para vetores de probabilidades. Normalmente, esta função é utilizada na camada de saída das redes neurais MLP, auxiliando na identificação das classes dos elementos de entrada.

Função de perda

A função de perda, também conhecida como função custo, verifica a discrepância entre as previsões e os dados reais de entrada, permitindo verificar a evolução do aprendizado do modelo durante o treinamento.

Alguns dos tipos mais utilizados nas redes neurais MLP são:

- Erro quadrático médio: utilizado em problemas de regressão.
- Entropia cruzada: utilizada em problemas de classificação.
- Entropia cruzada binária: variação da entropia cruzada, utilizada em problemas de classificação com apenas duas classes.
- Erro absoluto médio: utilizado também em problemas de regressão, calculando a média das diferenças absolutas entre as previsões e os valores de entrada.

Otimizadores

Utilizados para ajustar os pesos dos neurônios durante o treinamento do modelo.

- *Stochastic Gradient Descent* (SGD): atualiza os pesos de acordo com o gradiente negativo da função de perda.
- *Adaptive Moment Estimation* (Adam): trabalha com as médias móveis do gradiente. Eficaz em problemas de otimização com diversos tipos de dados.
- RMSprop: realiza a modulação da taxa de aprendizado, evitando oscilações e acelerando a convergência do modelo (ponto em que não haverá melhorias significativas de performance).
- *Adaptive Gradient Algorithm* (Adagrad): adapta a taxa de aprendizado para cada parâmetro de forma individual, diferente dos demais que realizam alterações em todos os pesos. Eficaz para pesos com escalas bastante diferentes entre si.

Métricas

As métricas são medidas quantitativas que visam avaliar o desempenho da rede durante o treinamento e, consequentemente, garantir que o percentual calculado para tal desempenho seja mantido (de maneira probabilística) após a validação e durante a utilização do modelo no futuro.

- Acurácia: métrica mais básica, utilizada amplamente. Mede a quantidade de acertos em relação à quantidade treinada.
- Precisão: mede a proporção de verdadeiros positivos em relação à soma de verdadeiros positivos e falsos positivos.
- *Recall*: conhecida como sensibilidade, mede a proporção de verdadeiros positivos em relação à soma de verdadeiros positivos e falso negativos.
- F1-Score: medida combinada de *precisão* e *recall*, fornecendo uma média entre estas duas métricas.

Épocas e amostras

Tanto para a rede neural MLP quanto para qualquer outro tipo de rede neural, o parâmetro época pode ser considerado como o número de rodadas que a rede executará dentro de um mesmo treinamento, e para cada uma destas rodadas, fará o treinamento em X amostras do percentual da base de dados selecionado.

Por exemplo, considere uma base de dados com 20 elementos (é um valor muito pequeno para treinamento de uma rede neural, mas para nosso exemplo didático, valores menores tornam mais fácil o entendimento). Antes de realizarmos o treinamento da rede MLP, configuramos o percentual de treinamento para 70% (14 elementos) e 30% (6 elementos) para testes posteriores de desempenho. O algoritmo vai dividir o conjunto total da base de dados (20 elementos) em dois subconjuntos, um com 14 elementos e outro com 6 elementos. Essa divisão pode acontecer de forma aleatória, sorteando elementos, ou simplesmente selecionando os 14 primeiros para um subconjunto e os 6 restantes para o outro (o método de seleção dependerá do algoritmo utilizado).

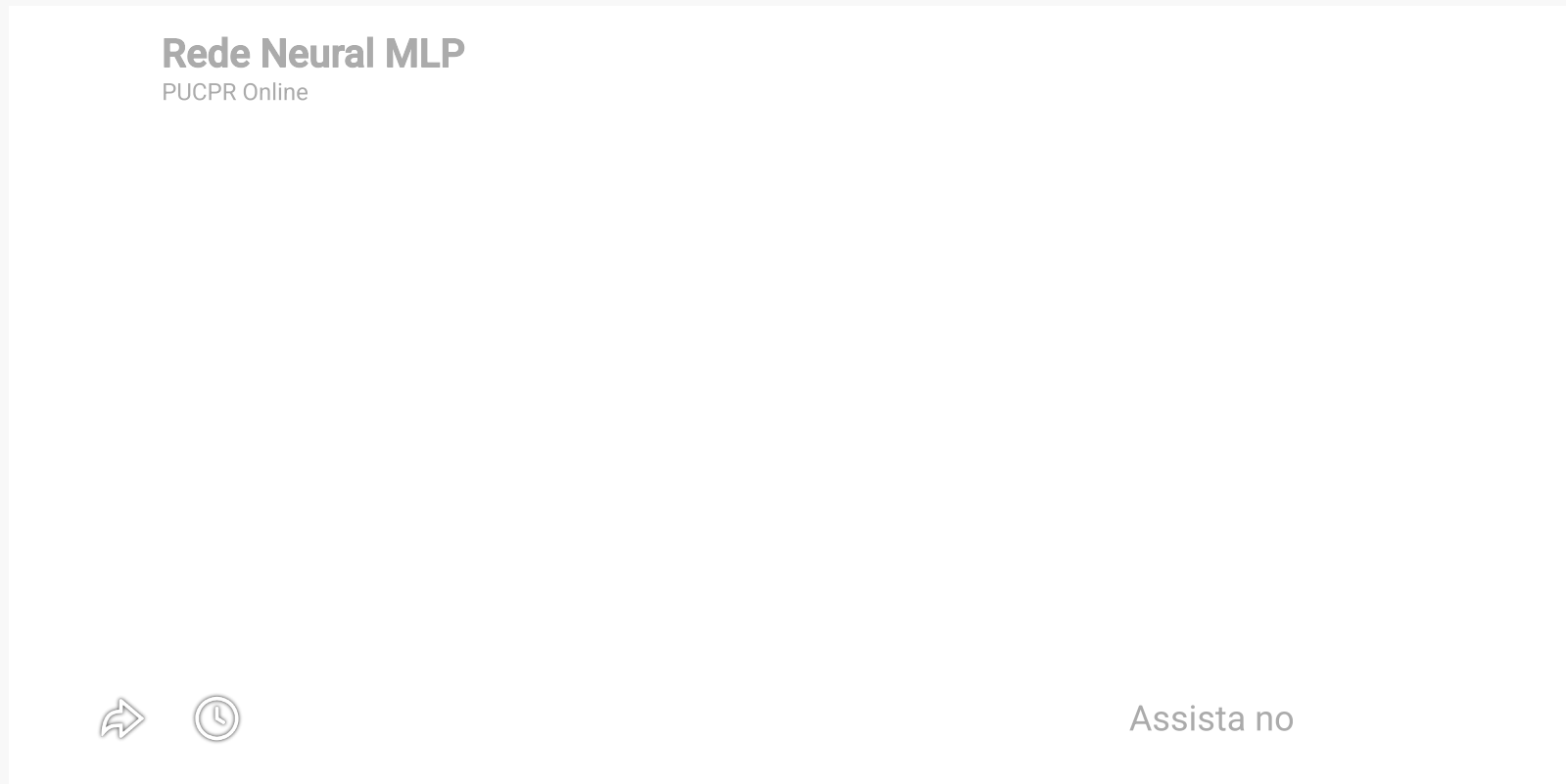
Agora, com o subconjunto de treinamento selecionado, vamos configurar, por exemplo, o número de épocas para 7 e o número de amostras para 2. Nesta situação, significa que ao rodarmos o treinamento da rede, o algoritmo executará 7 baterias de treinamento, cada uma delas com 2 elementos, passando, então, por todos os 14 elementos do subconjunto de treinamento. Os valores dos pesos (conexões entre os neurônios) serão atualizados ao final de cada bateria (época), de acordo com os valores das funções de perda (custo) e métricas de desempenho, visando sempre a melhorar o desempenho, reduzindo os erros de previsão.

No caso da rede neural MLP, para o problema de classificação de dígitos numéricos em figuras, vamos supor que o elemento de entrada seja uma figura com o número 2. Após o fluxo de treinamento percorrer todas as camadas iniciais e intermediárias (ou seja, realizados os cálculos dos pesos entre as conexões e os valores assumidos pelos neurônios), o fluxo passa então para a camada de saída, onde espera-se que o

neurônio 2 assuma valor igual a 1 (um), e todos os demais neurônios desta camada assumam valor igual a 0 (zero). Desta forma, significará que a rede reconheceu e identificou o dígito numérico da figura como 2 (acertando a previsão). A rede saberá que acertou, pois durante o treinamento, a informação de que realmente se trata do dígito 2 na figura é passado para a rede (isso só não acontece após o treinamento, nos momentos futuros em que a rede será então utilizada para realmente identificar dígitos sem o auxílio de um operador humano).

| VIDEOAULA (REDE NEURAL MLP)

Nesta videoaula, vamos abordar de forma breve o funcionamento referente ao treinamento de uma rede neural MLP, considerando os parâmetros ajustáveis da arquitetura do modelo.



Com as informações apresentadas até aqui, você estará apto para seguir em frente com as implementações dos modelos de redes neurais MLP, para a resolução de problemas específicos.

Agora, sua vez!

Realize a atividade formativa proposta para esta unidade e aprofunde seu conhecimento.

| Conclusão

Vimos nesta unidade as particularidades e funcionalidades de uma rede neural MLP. Conhecemos sua arquitetura e todos os seus parâmetros ajustáveis para treinamento e aprendizado, visando a melhorar o desempenho do modelo, aplicado em um problema de classificação.

Vimos, também, a importância de se escolher corretamente as Funções de ativação e de perda para que a *performance* fosse melhorada para esse tipo de problema de classificação. Além disso, discutimos a influência do número de neurônios da camada oculta (metade da soma dos neurônios de entrada e de saída), do percentual de dados para treinamento (70%) e teste (30%), e o número de épocas no treinamento do modelo. Valores fora dos padrões corretos fazem com que o modelo não apresente um bom desempenho, com função de perda muito alta e precisão muito baixa.

Possuímos agora, conhecimento suficiente para aplicar as redes neurais MLP em outros tipos de problemas de classificação, regressão etc., com um modelo capaz de realizar precisões com um nível suficientemente bom.

Nos vemos na sequência.

Até!

| Referência

HAYKIN, S. **Redes neurais**: Princípios e prática. 2. ed. Porto Alegre, RS: Bookman, 2007.

SILVA, F. M. *et al.* **Inteligência artificial**. Porto Alegre, RS: SAGAH, 2019.

FACELI, K. *et al.* **Inteligência artificial**: Uma abordagem de aprendizagem de máquina. Rio de Janeiro, RJ: LTC, 2011.



© PUCPR - Todos os direitos reservados.